Cyprus University of Technology

CEI 227 Προχωρημένη Αντικειμενοστρεφής Μεθοδολογία και Προγραμματισμός (UML, Java)

Γραφικά Περιβάλλοντα

Swing Library

Event-driven Programming

Παναγιώτης Ηλία

Swing

- Τα **GUIs** (**Graphical User Interfaces**) είναι τα συνηθισμένα interfaces που χρησιμοποιούν παράθυρα, κουμπιά, menus, κλπ
- Η **Swing** είναι βιβλιοθήκη της Java για τον προγραμματισμό τέτοιων interfaces
  - Η μετεξέλιξη του **AWT** (**Abstract Window Toolkit**) το οποίο ήταν το πρώτο αλλά όχι τόσο επιτυχημένο πακέτο της Java για GUI
  - Πλέον υπάρχουν πιο καινούργιες βιβλιοθήκες όπως η **JavaFX**, αλλά η Swing είναι «native Java» και πιο απλή για την εισαγωγή εννοιών
- Θα δούμε κυρίως:
  - AWT
  - Swing
  - MVC
  - Frames, Layouts, Components

AWT & Swing

- **Abstract Windowing Toolkit (AWT)**: Πρώτο library της Sun για δημιουργία cross-platform GUI κλάσεων
  - Αντιστοιχεί γενικό κώδικα Java στο GUI σύστημα του λειτουργικού συστήματος.
  - ***Προβλήματα:*** Είναι limited, βαρύ και δύσχρηστο

- **Swing**: Νεότερο library για GUI, με πιο πολλές δυνατότητες και πιο lightweight
  - Σχεδιάζει στοιχεία GUI pixel-by-pixel αντί να τα αναθέτει στο Λειτουργικό
  - ***Πλεονεκτήματα:*** Πιο πολλές δυνατότητες, συμβατότητα, Α/Σ σχεδίαση.

  - ***Πρόβλημα :***
    - Συνυπάρχουν και οι 2 βιβλιοθήκες στην Java
    - Είναι εύκολο να τις μπερδέψουμε
    - Πολλές φορές χρειάζεται να χρησιμοποιήσουμε και τις 2 βιβλιοθήκες

Event driven programming

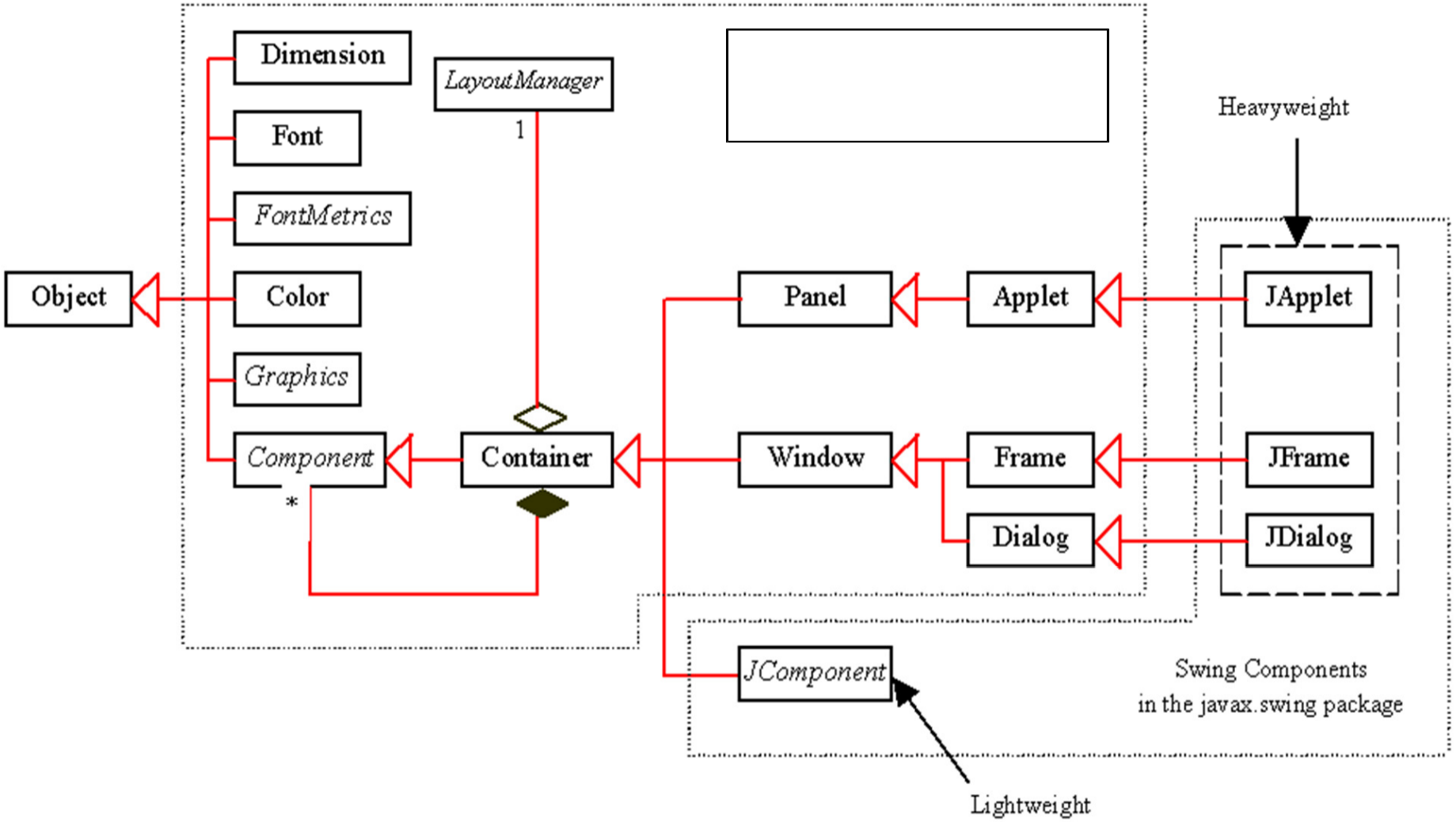
- Το **Swing** ακολουθεί το μοντέλο του **event-driven programming**
  - Υπάρχουν κάποια αντικείμενα που **πυροδοτούν συμβάντα** (firing an event)
  - Υπάρχουν άλλα αντικείμενα που ”ακούνε” (**listeners**) για συμβάντα
  - Αν προκληθεί ένα συμβάν υπάρχουν ειδικοί **χειριστές** του συμβάντος (**event handlers**) – μέθοδοι που χειρίζονται ένα συμβάν
  - Το **event** είναι κι αυτό ένα αντικείμενο το οποίο **μεταφέρει πληροφορία** μεταξύ του αντικειμένου που προκαλεί το συμβάν και του ακροατή
- Σας θυμίζουν κάτι όλα αυτά?
  - Παρόμοιες αρχές υπάρχουν στην δημιουργία και τον χειρισμό **εξαιρέσεων**

Swing

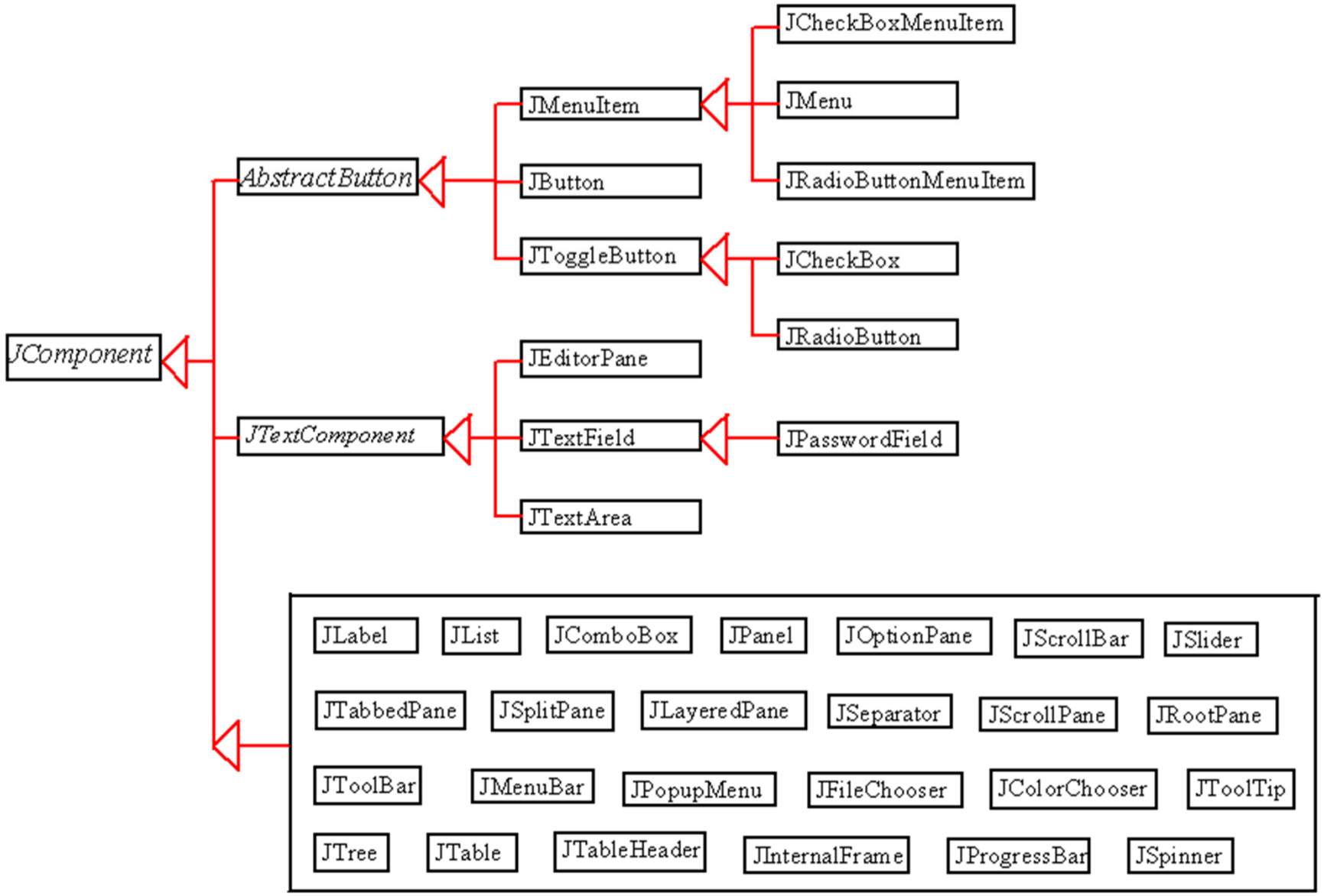


- Στην βιβλιοθήκη **Swing** ένα GUI αποτελείται από πολλά **στοιχεία** (**components**)
  - π.χ. παράθυρα, κουμπιά, μενού, κουτιά εισαγωγής κειμένου, κλπ.
- Τα components αυτά **πυροδοτούν συμβάντα**
  - π.χ. το πάτημα ενός κουμπιού, η εισαγωγή κειμένου, επιλογή σε ένα μενού κλπ
- Τα συμβάντα αυτά τα χειρίζονται τα **αντικείμενα listeners** που έχουν ειδικές μεθόδους γι αυτά
  - Τι γίνεται όταν πατάμε ένα κουμπί, όταν κάνουμε μια επιλογή κλπ
- Όλο το πρόγραμμα είναι μια αλληλουχία από **συμβάντα** και τον **χειρισμό** τους από τους ακροατές

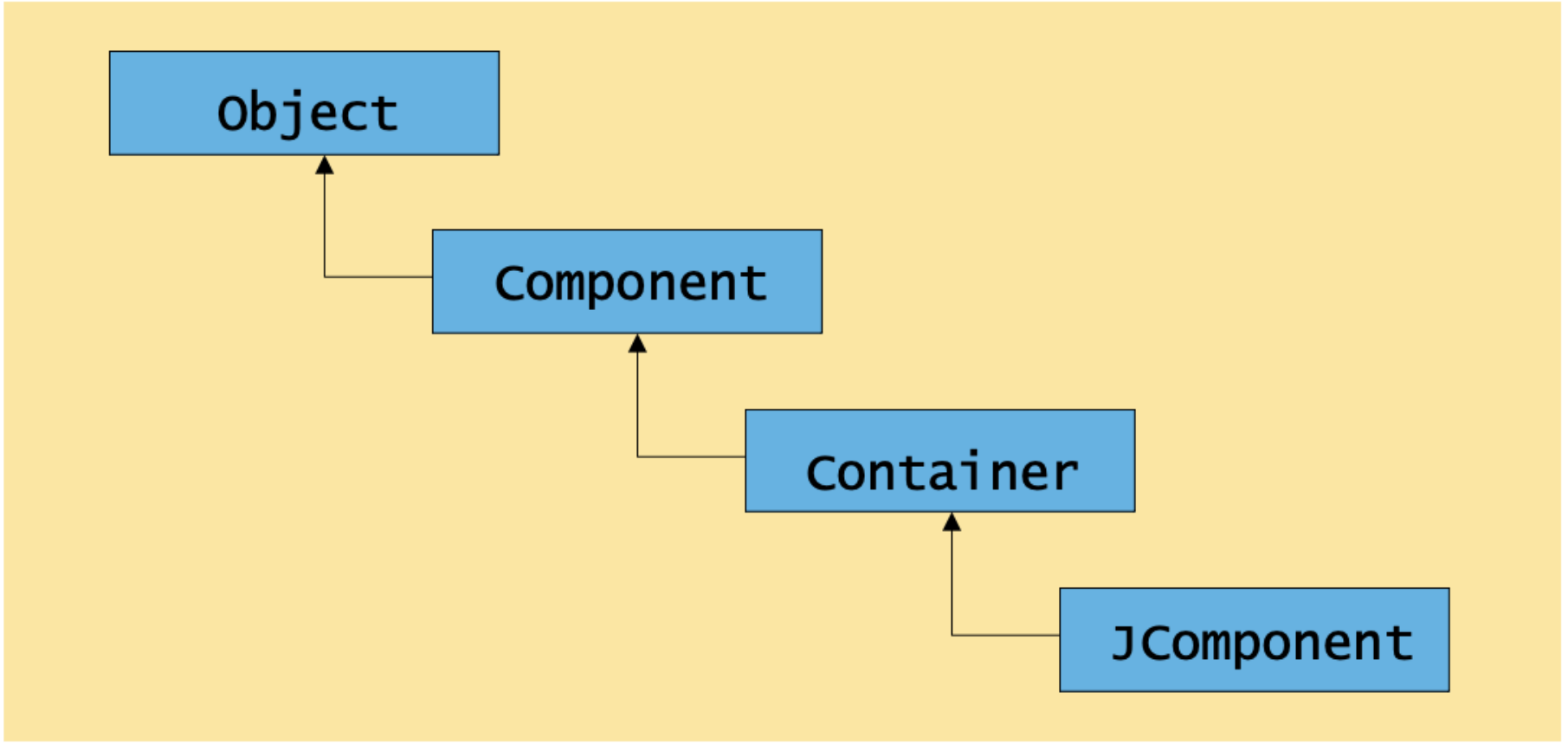
Swing Class Hierarchy



Συστατικά GUI (swing)



Superclasses of many of the Swing components



Overview of Swing components

- **Class Component**
  - Περιέχει την μέθοδο **paint** που σχεδιάζει το **Component** στην οθόνη
- **Class Container**
  - Συλλογή από σχετιζόμενα components
  - Περιέχει την μέθοδο **add** για την προσθήκη components
- **Class JComponent**
  - Superclass για τα πιο πολλά Swing components
  - *Pluggable look and feel* προσαρμογή/καθορισμό εμφάνισης
  - Παρέχει **keyboard shortcut keys** για τα components του GUI
  - Παρέχει μεθόδους για event-handling (άρα κοινός τρόπος διαχείρισης από subclasses)

Swing Class Hierarchy

- Component (AWT)
  - Window
    - Frame
      - **JFrame** (Swing)
      - **JDialog**
  - Container
    - JComponent (Swing)
      - **JButton**
      - **JComboBox**
      - **JMenuBar**
      - **JPopupMenu**
      - **JScrollPane**
      - **JSplitPane**
      - **JToolbar**
      - **JTextField**
      - **JColorChooser**
      - **JLabel**
      - **JOptionPane**
      - **JProgressBar**
      - **JSlider**
      - **JTabbedPane**
      - **JTree**
      - ...
      - **JFileChooser**
      - **JList**
      - **JPanel**
      - **JScrollbar**
      - **JSpinner**
      - **JTable**
      - **JTextArea**

```
import java.awt.*;
import javax.swing.*;
```

Component Properties

- Each has a **`get`** (or **`is`**) accessor and a **`set`** modifier method.
- examples: **`getColor`**, **`setFont`**, **`setEnabled`**, **`isVisible`**

name	type	description
background	`Color`	background color behind component
border	`Border`	border line around component
enabled	`boolean`	whether it can be interacted with
focusable	`boolean`	whether key text can be typed on it
font	`Font`	font used for text in component
foreground	`Color`	foreground color of component
height, width	`int`	component's current size in pixels
visible	`boolean`	whether component can be seen
tooltip text	`String`	text shown when hovering mouse
size, minimum / maximum / preferred size	`Dimension`	various sizes, size limits, or desired sizes that the component may take

JFrame



JFrame

Παράθυρο που κρατάει άλλα components

- `public JFrame()`
  `public JFrame(String title)`

Δημιουργεί παράθυρο με τίτλο

  - Η μέθοδος **`setVisible(true)`** κάνει το παράθυρο «ορατό» στην οθόνη μετά την δημιουργία του

- `public void add(Component comp)`
  Τοποθετεί το **component comp** μέσα στο παράθυρο

JFrame

Το JFrame ορίζει ένα βασικό απλό παράθυρο.
Ο παρακάτω κώδικας δημιουργεί ένα παράθυρο

```
import javax.swing.JFrame;

public class JFrameDemo
{
    public static final int WIDTH = 300;
    public static final int HEIGHT = 200;

    public static void main(String[] args)
    {
        JFrame firstWindow = new JFrame( );
        firstWindow.setSize(WIDTH, HEIGHT);

        firstWindow.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        firstWindow.setVisible(true);
    }
}
```

Καθορίζει το μέγεθος (πλάτος, ύψος) του παραθύρου μετρημένο σε pixels

Κάνει το παράθυρο ορατό

Καθορίζει τι κάνει το παράθυρο όταν πατάμε το κουμπί για κλείσιμο

JFrame

- Επιλογές για το **setDefaultCloseOperation**:
  - **EXIT_ON_CLOSE**: Καλεί την Sytem.exit() και σταματάει το πρόγραμμα
  - **DO_NOTHING_ON_CLOSE**: δεν κάνει τίποτα, ουσιαστικά δεν μας επιτρέπει να κλείσουμε το παράθυρο
  - **HIDE_ON_CLOSE**: Κρύβει το παράθυρο αλλά δεν σταματάει το πρόγραμμα
- Άλλες μέθοδοι:
  - **add**: προσθέτει ένα συστατικό (component) στο παράθυρο (π.χ. ένα κουμπί)
  - **setTitle(String)**: δίνει ένα όνομα στο παράθυρο που δημιουργούμε.

Ετικέτες - JLabel



JLabel

- Αφού έχουμε φτιάξει το βασικό παράθυρο μπορούμε πλέον να αρχίσουμε να **προσθέτουμε components**
- Μπορούμε να προσθέσουμε ένα (σύντομο) κείμενο στο παράθυρο μας προσθέτοντας μια **ετικέτα** (**label**)
- **JLabel** class: μας επιτρέπει να δημιουργήσουμε μια ετικέτα με συγκεκριμένο κείμενο
  - `JLabel greeting = new JLabel("Hello World!");`
- Αφού δημιουργήσουμε το **label** το **προσθέτουμε** μέσα στο **JFrame** μας
  - Καλούμε την μέθοδο **add** της **JFrame**

Frame με label

```
import javax.swing.JFrame;
import javax.swing.JLabel;

public class JLabelDemo
{
    public static final int WIDTH = 300;
    public static final int HEIGHT = 200;

    public static void main(String[] args)
    {
        JFrame firstWindow = new JFrame( );
        firstWindow.setSize(WIDTH, HEIGHT);

        firstWindow.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JLabel label = new JLabel("Hello World!");
        firstWindow.add(label);

        firstWindow.setVisible(true);
    }
}
```

Δημιουργία label με την κλάση JLabel και προσθήκη στο frame

Jlabel – Παράδειγμα 2

```
1       import java.awt.*;
2       import java.awt.event.*;
3       import javax.swing.*;
4
5       public class LabelTest extends JFrame {
6          private JLabel label1, label2, label3;
7
8          public LabelTest()
9          {
10           super( "Testing JLabel" );
11
12           // get content pane and set its layout
13           Container container = getContentPane();
14           container.setLayout( new FlowLayout() );
15
16           // JLabel constructor with a string argument
17           label1 = new JLabel( "Label with text" );
18           label1.setToolTipText( "This is label1" );
19           container.add( label1 );
```

Δημιουργεί το πρώτο **JLabel** με text “Label with text”

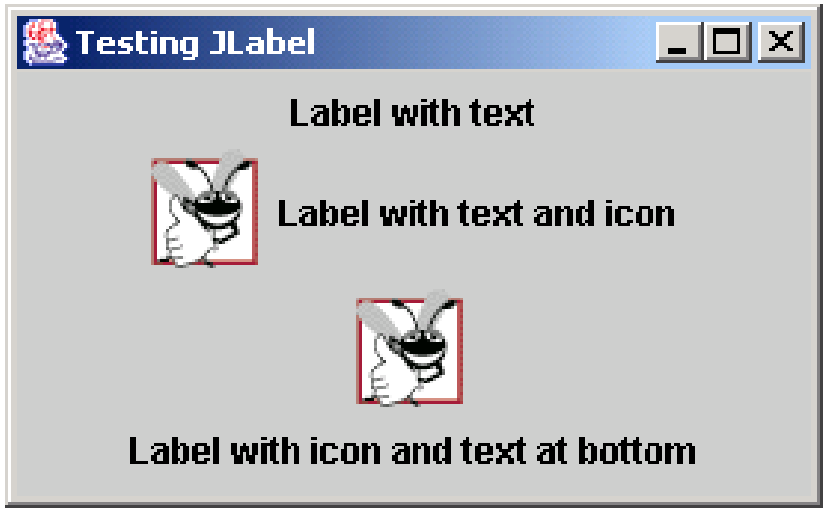
Tool tip είναι **text** που εμφανίζεται όταν μετακινούμε τον κέρσορα πάνω από το **JLabel**

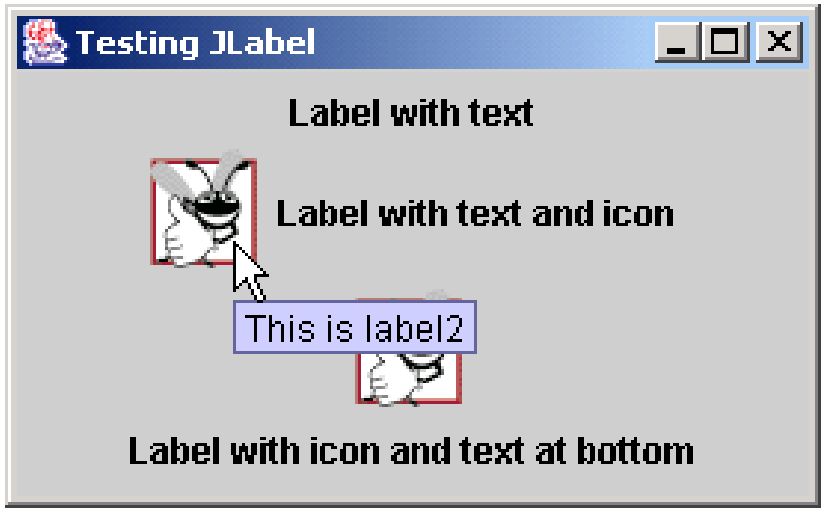
Συνέχεια στην επόμενη

```
20      // JLabel constructor with string, Icon and alignment arguments
21      Icon bug = new ImageIcon( "bug1.gif" );
22      label2 = new JLabel( "Label with text and icon", bug, SwingConstants.LEFT );
23      label2.setToolTipText( "This is label2" );
24      container.add( label2 );
25
26      // JLabel constructor no arguments
27      label3 = new JLabel();
28      label3.setText( "Label with icon and text at bottom" );
29      label3.setIcon( bug );
30      label3.setHorizontalTextPosition( SwingConstants.CENTER );
31      label3.setVerticalTextPosition( SwingConstants.BOTTOM );
32      label3.setToolTipText( "This is label3" );
33      container.add( label3 );
34
35      setSize( 275, 170 );
36      setVisible( true );
37
38      } // end constructor
39
40      public static void main( String args[] ){
41           LabelTest application = new LabelTest();
42           application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
43      }
```

Δημιουργεί το δεύτερο **JLabel** με εικόνα αριστερά από το κείμενο

Δημιουργεί τρίτο **JLabel** με κείμενο κάτω από την εικόνα


Testing JLabel
Label with text
Label with text and icon
Label with icon and text at bottom


Testing JLabel
Label with text
Label with text and icon
This is label2
Label with icon and text at bottom

Buttons - Κουμπιά



JButton

- Ένα άλλο component για ένα γραφικό περιβάλλον είναι τα **κουμπιά**
- Δημιουργούμε κουμπιά με την κλάση **JButton**
  - `JButton button = new JButton("click me");`
  - Το κείμενο στον constructor είναι αυτό που εμφανίζεται **πάνω** στο κουμπί
- Για να ξέρουμε τι κάνει το κουμπί όταν πατηθεί θα πρέπει να συνδέσουμε το κουμπί με ένα **listener**
  - Ο **listener** είναι ένα αντικείμενο μιας κλάσης που υλοποιεί το **interface ActionListener** η οποία έχει την μέθοδο
    - `actionPerformed(ActionEvent e)`: χειρίζεται ένα συμβάν
  - Αφού δημιουργήσουμε το αντικείμενο του listener το **συνδέουμε** με το **κουμπί** χρησιμοποιώντας την μέθοδο της **JButton**:
    - `addActionListener(ActionListener)`

Παράθυρο με κουμπί

```
import javax.swing.JFrame;
import javax.swing.JButton;

public class ButtonDemo
{
    public static final int WIDTH = 300;
    public static final int HEIGHT = 200;

    public static void main(String[] args)
    {
        JFrame firstWindow = new JFrame( );
        firstWindow.setSize(WIDTH, HEIGHT);

        firstWindow.setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);

        JButton endButton = new JButton("Click to end program.");

        EndingListener buttonEar = new EndingListener( );
        endButton.addActionListener(buttonEar);

        firstWindow.add(endButton);

        firstWindow.setVisible(true);
    }
}
```

Δημιουργία κουμπιού με την κλάση JButton

Δημιουργία και καταχώριση του listener στο κουμπί

Προσθήκη κουμπιού στο frame

```
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;

public class EndingListener implements ActionListener
{
    public void actionPerformed(ActionEvent e)
    {
        System.exit(0);
    }
}
```

Ένας listener υλοποιεί το interface ActionListener και πρέπει να υλοποιεί την μέθοδο actionPerformed(ActionEvent)

- Όταν πατάμε το κουμπί στο GUI καλείται η μέθοδος **actionPerfomed** του **listener** που έχουμε **καταχωρίσει** για το κουμπί
- Η κλήση της **actionPerformed** από τον **ActionListener** γίνεται **αυτόματα** μέσω της βιβλιοθήκης Swing, δεν την κάνει ο προγραμματιστής
- Η παράμετρος **ActionEvent** περιέχει πληροφορία σχετικά με το **event** που μπορεί να χρησιμοποιηθεί

```
import javax.swing.JFrame;
import javax.swing.JButton;


public class FirstWindow extends JFrame
{
    public static final int WIDTH = 300;
    public static final int HEIGHT = 200;

    public FirstWindow( )
    {
        super( );
        setSize(WIDTH, HEIGHT);

        setTitle("First Window Class");

        setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);

        JButton endButton = new JButton("Click to end program.");
        endButton.addActionListener(new EndingListener( ));
        add(endButton);
    }
}
```

Πιο σωστός τρόπος να ορίσουμε το παράθυρο μας ως ένα τύπο παράθυρου που επεκτείνει την κλάση JFrame

Η δημιουργία του ActionListener γίνεται ως ανώνυμο αντικείμενο μιας και δεν θα το χρησιμοποιήσουμε ποτέ άμεσα

```
public class DemoButtonWindow
{
    public static void main(String[] args)
    {
        FirstWindow w = new FirstWindow( );
        w.setVisible(true);
    }
}
```

Εδώ δημιουργούμε το παράθυρο μας

Αυτό είναι και το σωστό σημείο να αποφασίσουμε αν το παράθυρο θα είναι visible ή όχι.

Buttons – Παράδειγμα 2

```
import java.awt.*;          // Where is the other button?
import javax.swing.*;

public class GuiExample1 {
    public static void main(String[] args) {
        JFrame frame = new JFrame();
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(new Dimension(300, 100));
        frame.setTitle("A frame");

        JButton button1 = new JButton();
        button1.setText("I'm a button.");
        button1.setBackground(Color.BLUE);
        frame.add(button1);

        JButton button2 = new JButton();
        button2.setText("Click me!");
        button2.setBackground(Color.RED);
        frame.add(button2);

        frame.setVisible(true);
    }
}
```



Text Box

- Μπορούμε να δημιουργήσουμε ένα πεδίο κειμένου με την κλάση **`JTextField`**
  - Το **JTextField** δημιουργεί ένα **text box** μίας γραμμής
  - **`getText()`** : με την εντολή αυτή **διαβάζουμε** το κείμενο που δόθηκε σαν είσοδος στο text box
  - **`setText(String)`** : με την εντολή αυτή **θέτουμε** το κείμενο στο text box
  - **`setEditable(false)`**, ο χρήστης δεν μπορεί να τροποποιήσει το κείμενο
- Για ένα πεδίο κειμένου μεγαλύτερο από μία γραμμή μπορούμε να χρησιμοποιήσουμε την κλάση **JTextArea**

Παράδειγμα

```
JTextField name = new
JTextField(NUMBER_OF_CHAR);

namePanel.add(name,
BorderLayout.SOUTH);


JButton actionButton = new
JButton("Click me");


actionButton.addActionListener(this);

buttonPanel.add(actionButton);


JButton clearButton = new
JButton("Clear");

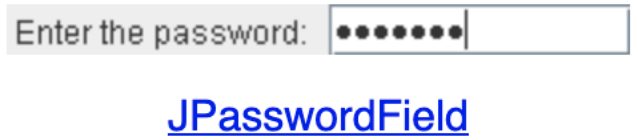
clearButton.addActionListener(this);

buttonPanel.add(clearButton);
```

```
public void actionPerformed(ActionEvent e)
{
    String actionCommand = e.getActionCommand( );

    if (actionCommand.equals("Click me"))
        name.setText("Hello " + name.getText( ));
    else if (actionCommand.equals("Clear"))
        name.setText("");
    else
        name.setText("Unexpected error.");
}
```

Text Box



- `JPasswordField`
  - Extends `JTextField`
  - Κρύβει τους χαρακτήρες που εισάγει ο χρήστης

Text Boxes – Μεγάλο Παράδειγμα

```
1       import java.awt.*;
2       import java.awt.event.*;
3       import javax.swing.*;
4
5       public class TextFieldTest extends JFrame {
6          private JTextField textField1, textField2, textField3;
7          private JPasswordField passwordField;
8
9          public TextFieldTest()
10         {
11             super( "Testing JTextField and JPasswordField" );
12
13             Container container = getContentPane();
14             container.setLayout( new FlowLayout() );
15
16             // construct textfield with default sizing
17             textField1 = new JTextField( 10 );
18             container.add( textField1 );
19
20             // construct textfield with default text
21             textField2 = new JTextField( "Enter text here" );
22             container.add( textField2 );
```

Το textField περιέχει ένα κενό String

Συνέχεια στην επόμενη

```
24          // construct textfield with default text,
25          // 20 visible elements and no event handler
26          textField3 = new JTextField( "Uneditable text field", 20 );
27          textField3.setEditable( false );
28          container.add( textField3 );
29
30          // construct passwordfield with default text
31          passwordField = new JPasswordField( "Hidden text" );
32          container.add( passwordField );
33
34          // register event handlers
35          TextFieldHandler handler = new TextFieldHandler();
36          textField1.addActionListener( handler );
37          textField2.addActionListener( handler );
38          textField3.addActionListener( handler );
39          passwordField.addActionListener( handler );
40
41          setSize( 325, 100 );
42          setVisible( true );
43
44       } // end constructor TextFieldTest
45
46       public static void main( String args[] )
47       {
48          TextFieldTest application = new TextFieldTest();
49          application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
50       }
```

Το textField περιέχει text που δεν μπορεί να αλλαχτεί

Καταχωρεί (register) GUI components with **TextFieldHandler** (register for **ActionEvents**)

Συνέχεια στην επόμενη

```
52       // private inner class for event handling
53       private class TextFieldHandler implements ActionListener {
54
55          // process textfield events
56          public void actionPerformed( ActionEvent event )
57          {
58             String string = "";
59
60             // user pressed Enter in JTextField textField1
61             if ( event.getSource() == textField1 )
62                string = "textField1: " + event.getActionCommand();
63
64             // user pressed Enter in JTextField textField2
65             else if ( event.getSource() == textField2 )
66                string = "textField2: " + event.getActionCommand();
67
68             // user pressed Enter in JTextField textField3
69             else if ( event.getSource() == textField3 )
70                string = "textField3: " + event.getActionCommand();
71
72             // user pressed Enter in JTextField passwordField
73             else if ( event.getSource() == passwordField ) {
74                string = "passwordField: " +
75                   new String( passwordField.getPassword() );
76             }
77
78             JOptionPane.showMessageDialog( null, string );
79
80          } // end method actionPerformed
81       } // end private inner class TextFieldHandler
82    } // end class TextFieldTest
```

Κάθε στιγμιότυπο του **TextFieldHandler** είναι **ActionListener**

Η μέθοδος **actionPerformed** εκτελείται όταν ο χρήστης πατήσει Enter



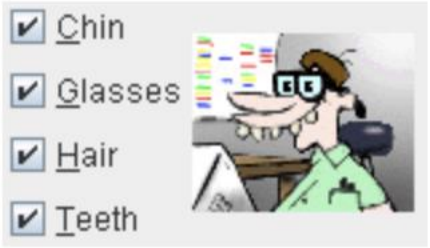
JSlider



JSpinner



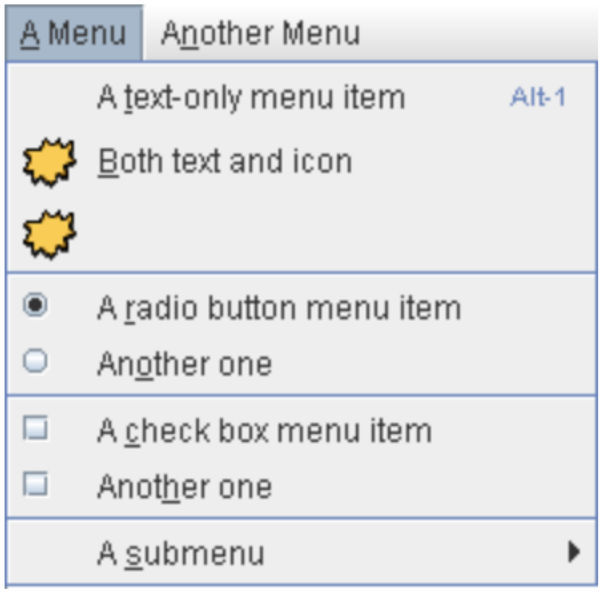
JTextField



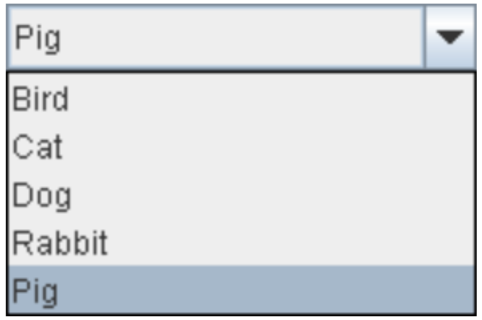
JCheckBox



JRadioButton



JMenu

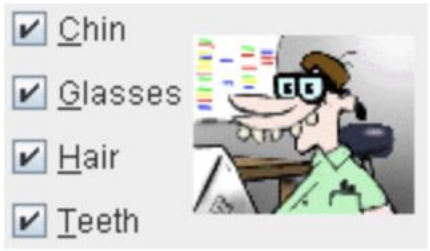


JComboBox



JList

JCheckBox και JRadioButton



JCheckBox



JRadioButton

- **State buttons**
  - **On/Off** ή **true/false**
  - 3 τύποι
    - **JToggleButton**
    - **JCheckBox**
    - **JRadioButton**
  - **getSource** επιστρέφει το button object που δημιούργησε το event
  - **isSelected** μας λέει αν το JCheckBox επιλέγηκε ή όχι
  - **JRadioButtons** παρουσιάζεται σαν ομάδα από Buttons
  - **JRadioButtons** είναι mutually exclusive (μπορεί να επιλεγεί 1 option μόνο)

Πολλά components - Layout

- Αν θέλουμε να βάλουμε **πολλά components** μέσα στο παράθυρο μας τότε θα πρέπει να προσδιορίσουμε **που** θα τοποθετηθούν αλλιώς θα μπούνε το ένα πάνω στο άλλο

- Αυτό γίνεται με την μέθοδο **setLayout** που καθορίζει την τοποθέτηση μέσα στο παράθυρο
  - Αυτό μπορεί να γίνει με διαφορετικούς τρόπους

JFrame as container

- Το **JFrame** είναι container. Τα Containers έχουν τις πιο κάτω μεθόδους:
  - **`public void add(Component comp)`**
  - **`public void add(Component comp, Object info)`**
    - Βάζει το **component** στο **container**
  - **`public void remove(Component comp)`**
  - **`public void setLayout(LayoutManager mgr)`**
    - Το Layout που θα χρησιμοποιηθεί για να βάλουμε το το **component** στο **container**
  - **`public void validate()`**
    - Ανανεώνει το layout (αν έχει αλλάξει αφού το container εμφανίστηκε στην οθόνη)

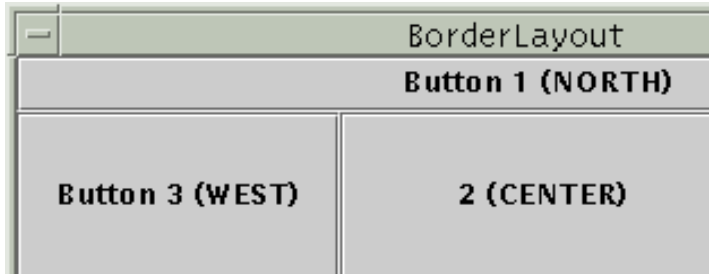
Μέγεθος των components

- Υπάρχει «προτιμώμενο» μέγεθος για τα αντικείμενα της Swing (*preferred size*)
  - Το κάθε αντικείμενο έχει το δικό του «προτιμώμενο» μέγεθος
  - Τόσο ώστε να χωράει τα περιεχόμενα του (text, icons, etc.)
- Κάποια layouts όπως το **FlowLayout** χρησιμοποιούν το **preferred size** για τα components που περιλαμβάνουν
- Κάποια άλλα όπως **BorderLayout** και **GridLayout** δεν λαμβάνουν υπόψη το **preferred size**, και χρησιμοποιούν διαφορετικό μέγεθος

Buttons at preferred size:



Not preferred size:



FlowLayout

- Απλά τοποθετεί τα components το ένα μετά το άλλο από τα αριστερά προς τα δεξιά
- Καλούμε την μέθοδο

```
setLayout(new FlowLayout());
```

(Πρέπει να έχουμε κάνει **import java.awt.FlowLayout**)

- Μετά προσθέτουμε κανονικά τα components με την **add**

BorderLayout

- Στην περίπτωση αυτή ο χώρος χωρίζεται σε πέντε περιοχές: **North**, **South**, **East**, **West, Center**
- Καλούμε την μέθοδο

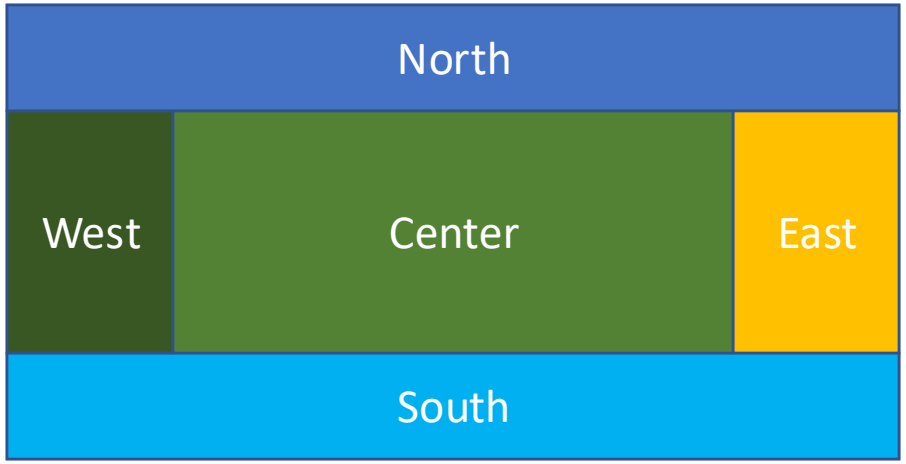
```
setLayout(new BorderLayout());
```

(Πρέπει να έχουμε κάνει **import java.awt.BorderLayout**)

- Όταν προσθέτουμε τα components με την add, προσδιορίζουμε την περιοχή στην οποία θα προστεθούν

Παράδειγμα:

- `add(label, BorderLayout.CENTER)`



GridLayout

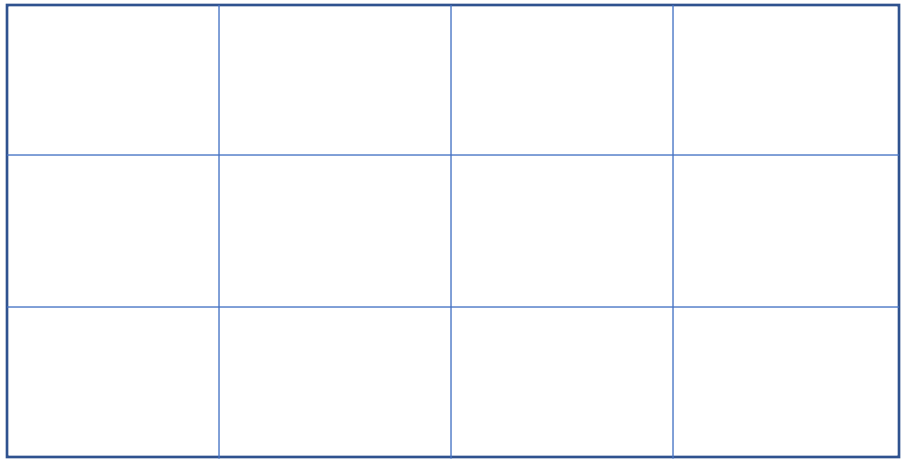
- Στην περίπτωση αυτή ορίζουμε ένα πλέγμα με **n** γραμμές και **m** στήλες και αυτό γεμίζει από τα αριστερά προς τα δεξιά και από πάνω προς τα κάτω
- Καλούμε την εντολή

```
setLayout(new GridLayout(n,m));
```

(Πρέπει να έχουμε κάνει **include java.awt.GridLayout**)

- Μετά προσθέτουμε κανονικά τα components με την **add**

Grid 3x4



Παράδειγμα

- Δημιουργείστε ένα παράθυρο με τρία κουμπιά:
  - Το ένα κάνει το χρώμα του παραθύρου μπλε, το άλλο κόκκινο και το τρίτο κλείνει το παράθυρο
  - Κώδικας: **MultiButtonWindow**

```
import javax.swing.JFrame;
import javax.swing.JButton;
import javax.swing.JLabel;
import java.awt.Color;
import java.awt.FlowLayout;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;

public class MultiButtonWindow extends JFrame implements ActionListener
{
    public static final int WIDTH = 300;
    public static final int HEIGHT = 200;


    public MultiButtonWindow( )
    {
        super("Multi-Color");
        setSize(WIDTH, HEIGHT);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        setLayout(new FlowLayout());

        JLabel label = new JLabel("Pick A Color");
        add(label);

        JButton blueButton = new JButton("Blue");
        blueButton.addActionListener(this);
        add(blueButton);

        JButton redButton = new JButton("Red");
        redButton.addActionListener(this);
        add(redButton);

        JButton endButton = new JButton("Exit");
        endButton.addActionListener(this);
        add(endButton);
    }
```

Η κλάση υλοποιεί τον listener και την actionPerformed μεθοδο

Ορίζουμε τα χαρακτηριστικά του βασικού παραθύρου

Δημιουργούμε τα τρία κουμπιά και τα προσθέτουμε στο frame

Ο listener των κουμπιών είναι το ίδιο το αντικείμενο (this)

Συνέχεια στην επόμενη

Συνέχεια από την προηγούμενη

```
    public void actionPerformed(ActionEvent e)
    {
        String buttonType = e.getActionCommand( );

        switch (buttonType){
            case "Blue":
                getContentPane().setBackground(Color.BLUE);
                break;
            case "Red":
                getContentPane().setBackground(Color.RED);
                break;
            case "Exit":
                System.exit(0);
        }
    }


    public static void main(String[] args)
    {
        MultiButtonWindow w = new MultiButtonWindow();
        w.setVisible(true);
    }

}
```

Η actionPerformed που καλείται όταν πατηθούν τα κουμπιά (μιας και το αντικείμενο είναι και listener)

Επιστρέφει το actionCommand String, το οποίο αν δεν το έχουμε αλλάξει είναι το όνομα του κουμπιού

Το αποτέλεσμα του κάθε διαφορετικού κουμπιού

Η `getContentPane` μας δίνει πρόσβαση στα χαρακτηριστικα του frame.
Η `setBackground` αλλάζει το χρώμα του frame

Δημιουργία του παραθύρου

Αξιοσημείωτα

- `public class MultiButtonWindow extends JFrame implements ActionListener`
  - Μπορούμε να κάνουμε τον **listener** να είναι **το ίδιο το παράθυρο**, αυτό θα αναλάβει να υλοποιήσει τη μέθοδο **actionPerformed**.
  - Όταν καταχωρούμε τον listener:

    ```
    blueButton.addActionListener(this);
    ```

Αξιοσημείωτα

- `getContentPane().setBackground(Color.BLUE);`
  - Αλλάζει το background χρώμα του παραθύρου.
  - Η κλάση **Color** μας δίνει τα χρώματα

- `String buttonType = e.getActionCommand();`
  - Με την εντολή αυτή παίρνουμε το String το οποίο δώσαμε σαν τίτλο στο κουμπί

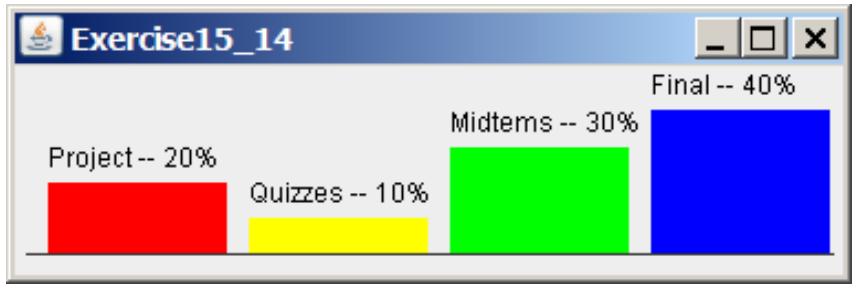
actionCommand

- Ένα String πεδίο που κρατάει πληροφορία για το συμβάν
  - Αν δεν αλλάξουμε κάτι αυτό είναι το όνομα του κουμπιού
- Μπορούμε να διαβάσουμε το String με την εντολή **getActionCommand**
- Μπορούμε να θέσουμε μια τιμή στο String με την εντολή **setActionCommand(String)**
- Π.χ. `redButton.setActionCommand("RedButtonClick");`

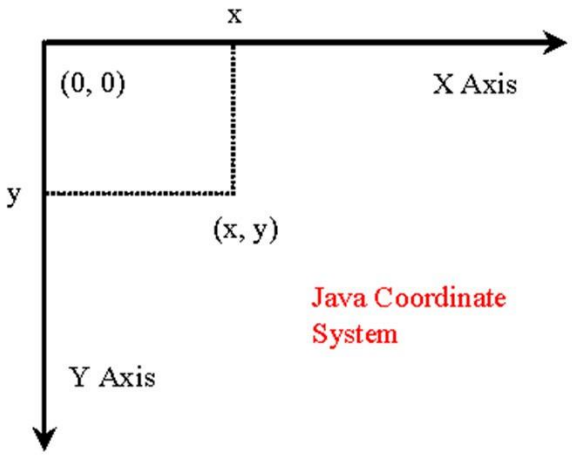
Χρώματα

- Μπορούμε να ορίσουμε τα δικά μας χρώματα με την **RGB** σύμβαση
  - `Color myColor = new Color(200,100,4);`
  - Τα ορίσματα είναι οι RGB (**Red, Green, Blue**) τιμές

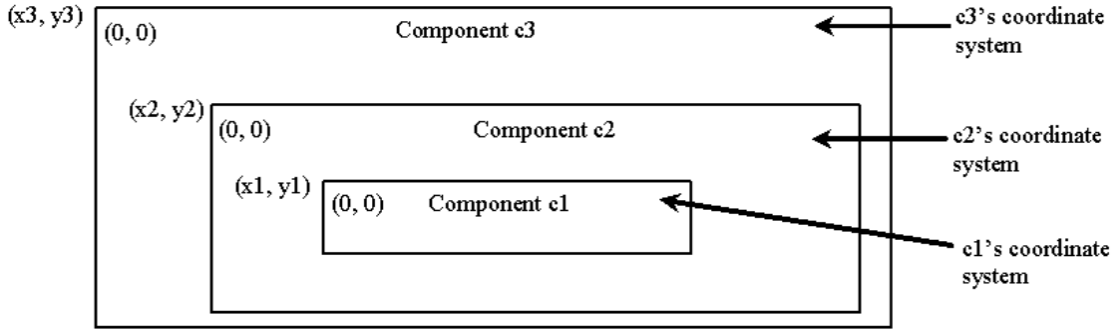
Custom Graphics

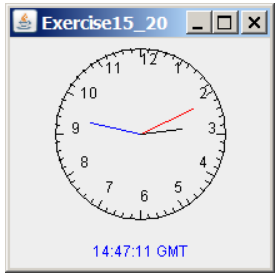


- Πως σχεδιάζουμε αυθαίρετα (custom) γραφικά;
  - Με την χρήση της κλάσης java.awt.Graphics
  - Παρέχει μεθόδους για σχεδίαση γραμμών, ορθογώνιων, κτλ.
  - Σύστημα Συντεταγμένων JAVA



Σύστημα Συντεταγμένων σε αντικείμενα

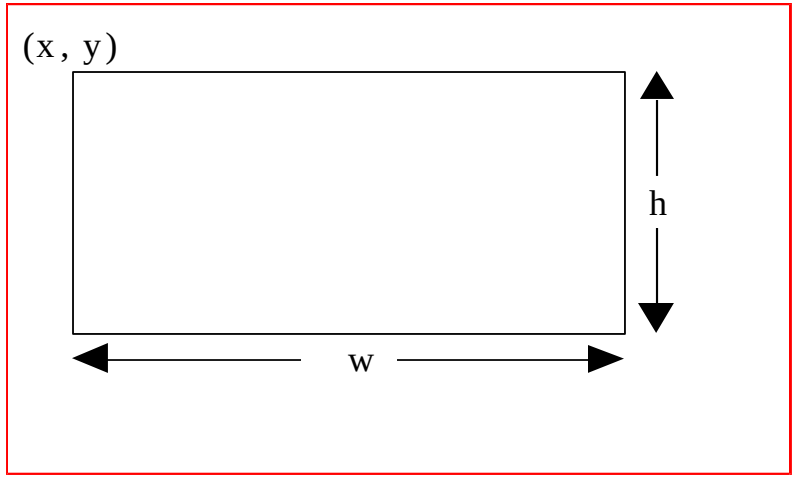


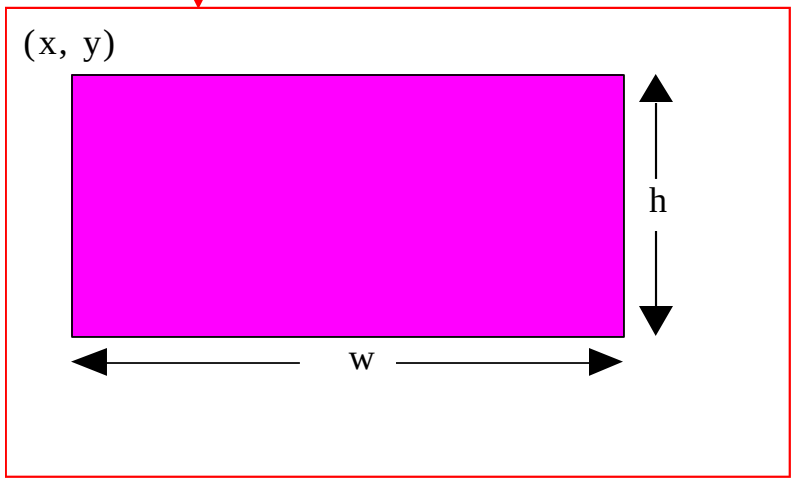




```
drawRect(int x, int y, int w, int h);
```

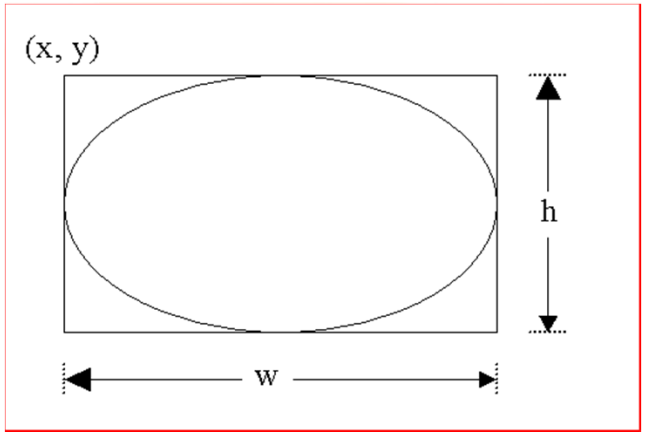
```
fillRect(int x, int y, int w, int h);
```






```
drawOval(int x, int y,
         int w, int h);
```

```
fillOval(int x, int y,
         int w, int h);
```



JPanel

```
setLayout(new BorderLayout());

JPanel buttonPanel = new JPanel();
buttonPanel.setLayout(new FlowLayout());

JButton button1 = new JButton("one");
buttonPanel.add(button1);

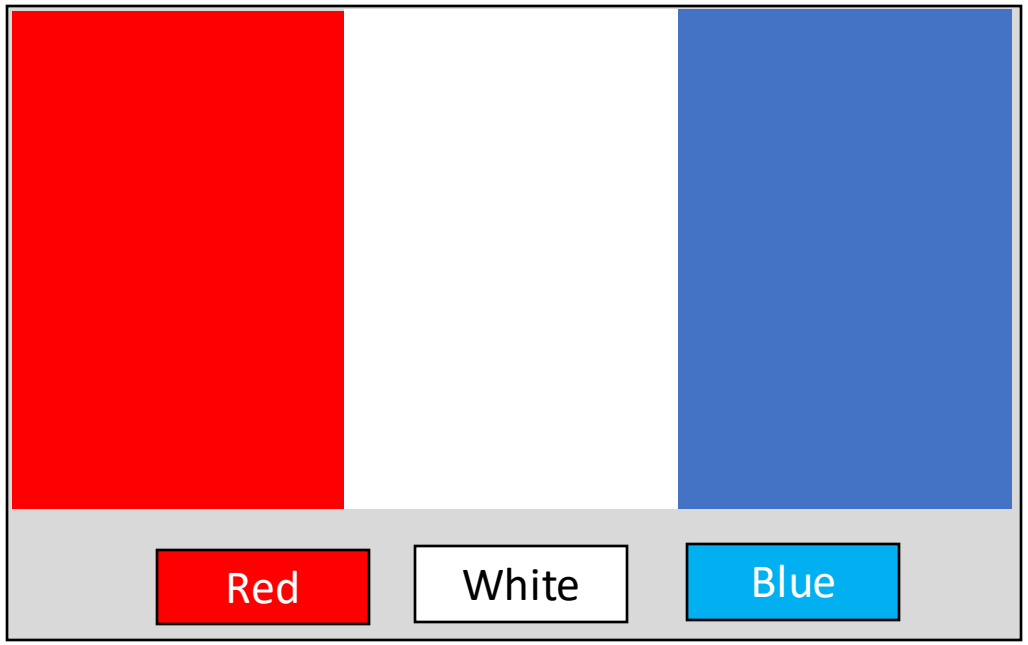
JButton button2 = new JButton("two");
buttonPanel.add(button2);

add(buttonPanel, BorderLayout.SOUTH);
```

- Το **panel** είναι ένα **container**
  - Μέσα σε ένα container μπορούμε να βάλουμε components και να ορίσουμε χειρισμό συμβάντων
- Τα panels κατά μία έννοια ορίζουν ένα **παράθυρο μέσα στο παράθυρο**
  - Το **panel** έχει κι αυτό το δικό του **layout** και τοποθετούμε μέσα σε αυτό συστατικά
  - Π.χ., ο παρακάτω κώδικας εκτελείται μέσα σε ένα **JFrame**

Παράδειγμα

- Θα δημιουργήσουμε ένα παράθυρο με τρία panels. Το κάθε panel θα παίρνει διαφορετικό χρώμα με ένα διαφορετικό κουμπί.



```
import javax.swing.JFrame;
import javax.swing.JPanel;
import java.awt.BorderLayout;
import java.awt.GridLayout;
import java.awt.FlowLayout;
import java.awt.Color;
import javax.swing.JButton;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;

public class PanelDemo extends JFrame implements ActionListener
{
    public static final int WIDTH = 300;
    public static final int HEIGHT = 200;

    private JPanel redPanel;
    private JPanel whitePanel;
    private JPanel bluePanel;


    public PanelDemo( )
    {
        super("Panel Demonstration");
        setSize(WIDTH, HEIGHT);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout( ));
```

Η κλάση υλοποιεί τον **listener** και την **actionPerformed** μεθοδο

Δηλώνουμε τα τρία πάνελ με τα τρία χρώματα

Ορίζουμε τα χαρακτηριστικά του βασικού παραθύρου

Συνέχεια στην επόμενη

Συνέχεια από την προηγούμενη

```
JPanel biggerPanel = new JPanel( );
biggerPanel.setLayout(new GridLayout(1, 3));

redPanel = new JPanel( );
redPanel.setBackground(Color.LIGHT_GRAY);
biggerPanel.add(redPanel);

whitePanel = new JPanel( );
whitePanel.setBackground(Color.LIGHT_GRAY);
biggerPanel.add(whitePanel);

bluePanel = new JPanel( );
bluePanel.setBackground(Color.LIGHT_GRAY);
biggerPanel.add(bluePanel);

add(biggerPanel, BorderLayout.CENTER);
```

Δημιουργούμε ένα μεγάλο πάνελ που θα κρατάει τα τρία χρωματιστά πάνελ

Δημιουργούμε τα χρωματιστά πάνελ και τα προσθέτουμε στο μεγάλο πάνελ

Βάζουμε το μεγάλο πάνελ στο κέντρο του παραθύρου

Συνέχεια στην επόμενη

Συνέχεια από την προηγούμενη

```
        JPanel buttonPanel = new JPanel( );
        buttonPanel.setBackground(Color.LIGHT_GRAY);
        buttonPanel.setLayout(new FlowLayout( ));

        JButton redButton = new JButton("Red");
        redButton.setBackground(Color.RED);
        redButton.addActionListener(this);
        buttonPanel.add(redButton);

        JButton whiteButton = new JButton("White");
        whiteButton.setBackground(Color.WHITE);
        whiteButton.addActionListener(this);
        buttonPanel.add(whiteButton);

        JButton blueButton = new JButton("Blue");
        blueButton.setBackground(Color.BLUE);
        blueButton.addActionListener(this);
        buttonPanel.add(blueButton);

        add(buttonPanel, BorderLayout.SOUTH);

    } // τέλος του constructor
```

Δημιουργούμε ένα **πάνελ** που θα κρατάει **τα τρία κουμπιά**

Δημιουργούμε τα τρία κουμπιά και τα προσθέτουμε στο πάνελ

Ο ακροατής των κουμπιών είναι το **ίδιο** το αντικείμενο

Βάζουμε το πάνελ με τα κουμπιά στον πάτο του παραθύρου

Συνέχεια στην επόμενη

Συνέχεια από την προηγούμενη

```
    public void actionPerformed(ActionEvent e)
    {
        String buttonString = e.getActionCommand( );

        if (buttonString.equals("Red"))
             redPanel.setBackground(Color.RED);
        else if (buttonString.equals("White"))
            whitePanel.setBackground(Color.WHITE);
        else if (buttonString.equals("Blue"))
            bluePanel.setBackground(Color.BLUE);
        else
            System.out.println("Unexpected error.");
    }

    public static void main(String[] args)
    {
        PanelDemo gui = new PanelDemo( );
        gui.setVisible(true);
    }

}
```

Η συνάρτηση **actionPerformed** που καλείται όταν πατηθούν τα κουμπιά (μιας και το αντικείμενο είναι και listener)

Επιστρέφει το **actionCommand** String, το οποίο αν δεν το έχουμε αλλάξει είναι το όνομα του κουμπιού

Το αποτέλεσμα του κάθε διαφορετικού κουμπιού.

Δημιουργία του παραθύρου

Menu

- **Drop-down menus:**
  - **`JMenuItem`**: κρατάει μία από τις επιλογές του menu
  - **`JMenu`**: κρατάει όλα τα JMenuItems
  - **`JMenuBar`**: κρατάει το Jmenu
  - **`setJMenuBar(JMenu)`** : θέτει το menu στην κορυφή του JFrame.

    Μπορούμε να χρησιμοποιήσουμε και τη γνωστή εντολή **`add`**.

Παράδειγμα

```
JMenu colorMenu = new JMenu("Add Colors");

JMenuItem redChoice = new JMenuItem("Red");
redChoice.addActionListener(this);
colorMenu.add(redChoice);

JMenuItem whiteChoice = new JMenuItem("White");
whiteChoice.addActionListener(this);
colorMenu.add(whiteChoice);

JMenuItem blueChoice = new JMenuItem("Blue");
blueChoice.addActionListener(this);
colorMenu.add(blueChoice);

JMenuBar bar = new JMenuBar( );
bar.add(colorMenu);
setJMenuBar(bar);
```

Δημιουργεί ένα drop-down menu

Δημιουργεί τις επιλογές του μενού και τις προσθέτει στο μενού

Δημιουργεί ένα menu bar στην κορυφή του παραθύρου και προσθέτει το menu σε αυτό

Pop-up Windows

- Αν θέλουμε να δημιουργήσουμε παράθυρα διαλόγου μπορούμε να χρησιμοποιήσουμε την κλάση **JOptionPane**
  - Πετάει (pops up) ένα παράθυρο το οποίο μπορεί να μας ζητάει είσοδο, ή να ζητάει επιβεβαίωση
  - Η δημιουργία και η διαχείριση των παραθύρων γίνεται με **στατικές μεθόδους**.

```
import javax.swing.JOptionPane;

public class PopUpDemo
{
  public static void main(String args[]){
    boolean done = false;
    while (!done){
      String classes = JOptionPane.showInputDialog("Enter number of classes");
      String students = JOptionPane.showInputDialog("Enter number of students");
      int totalStudents = Integer.parseInt(classes)*Integer.parseInt(students);

      JOptionPane.showMessageDialog(null, "Total number of students = "+totalStudents);


      int answer = JOptionPane.showConfirmDialog(null,
                   "Continue?",
                   "Confirm",
                    JOptionPane.YES_NO_OPTION);
      done = (answer == JOptionPane.NO_OPTION);
    }
    System.exit(0);
  }
}
```

Εμφανίζει ένα παράθυρο διαλόγου που ζητάει από τον χρήστη να δώσει είσοδο.
Η είσοδος αποθηκεύεται στο String που επιστρέφεται

Το αντικείμενο (component) που είναι πατέρας του pop-up, **null** η default τιμή

Εμφανίζει ένα παράθυρο που τυπώνει ένα μήνυμα

Η ερώτηση στο χρήστη

Τίτλος παραθύρου

Τύπος επιβεβαίωσης

Εμφανίζει ένα παράθυρο επιβεβαίωσης

Άλλοι τύποι επιβεβαίωσης:

- YES_OPTION
- OK_CANCEL_OPTION
- YES_NO_CANCEL_OPTION

Icons

- Μπορούμε να βάλουμε μέσα στο GUI μας και εικονίδια
- Παράδειγμα

Δημιουργεί ένα εικονίδιο από μία εικόνα

```
ImageIcon dukeIcon = new ImageIcon("duke_waving.gif");
JLabel dukeLabel = new JLabel("Mood check");
dukeLabel.setIcon(dukeIcon);


ImageIcon happyIcon = new ImageIcon("smiley.gif");
JButton happyButton = new JButton("Happy");
happyButton.setIcon(happyIcon);
```

Προσθέτει το εικονίδιο σε ένα label

Προσθέτει το εικονίδιο σε ένα button

Listeners

- Στο πρόγραμμα ορίσαμε την κλάση που δημιουργεί το παράθυρο (**extends JFrame**) να είναι και ο ακροατής (**implements ActionListener**) των events μέσα στο παράθυρο
  - Αυτό είναι μια βολική λύση γιατί όλος ο κώδικας είναι στο **ίδιο** σημείο
  - Έχει το πρόβλημα ότι έχουμε **μία μόνο** μέθοδο **actionPerformed** στην οποία θα πρέπει να ξεχωρίσουμε όλες τις περιπτώσεις

- Πιο βολικό να έχουμε ένα **διαφορετικό ActionListener** για κάθε event
  - **Προβλήματα**:
    - Θα πρέπει να ορίσουμε **πολλαπλές κλάσεις** listeners σε πολλαπλά αρχεία
    - Θα πρέπει να περνάμε σαν παραμέτρους τα στοιχεία που θέλουμε να αλλάξουμε

Listeners

- **Λύση**: Να ορίσουμε τους listeners που χρειάζεται το παράθυρο μας ως **εσωτερικές κλάσεις**
- **Υπενθύμιση**: μια εσωτερική κλάση ορίζεται μέσα σε μία άλλη κλάση και την βλέπει μόνο η κλάση που την ορίζει
- **Πλεονεκτήματα**:
  - Οι κλάσεις είναι πλέον **τοπικές** στον κώδικα που τις καλεί, μπορούμε να επαναχρησιμοποιούμε τα ίδια ονόματα
  - Οι κλάσεις έχουν πρόσβαση σε **ιδιωτικά πεδία**

```
JButton redButton = new JButton("Red");
redButton.setBackground(Color.RED);
redButton.addActionListener(new RedListener());
buttonPanel.add(redButton);

JButton whiteButton = new JButton("White");
whiteButton.setBackground(Color.WHITE);
whiteButton.addActionListener(new WhiteListener());
buttonPanel.add(whiteButton);

JButton blueButton = new JButton("Blue");
blueButton.setBackground(Color.BLUE);
blueButton.addActionListener(new BlueListener());
buttonPanel.add(blueButton);
```

Ορισμός των εσωτερικών κλάσεων-ακροατών (Κώδικας στο InnerListeners.java)

```
private class RedListener implements ActionListener{
    public void actionPerformed(ActionEvent e)
    {
        redPanel.setBackground(Color.RED);
    }
}

private class WhiteListener implements ActionListener{
    public void actionPerformed(ActionEvent e)
    {
        whitePanel.setBackground(Color.WHITE);
    }
}

private class BlueListener implements ActionListener{
    public void actionPerformed(ActionEvent e)
    {
        bluePanel.setBackground(Color.BLUE);
    }
}
```

Οι εσωτερικές κλάσεις έχουν πρόσβαση στα ιδιωτικά αντικείμενα πάνελ

Ανώνυμες κλάσεις

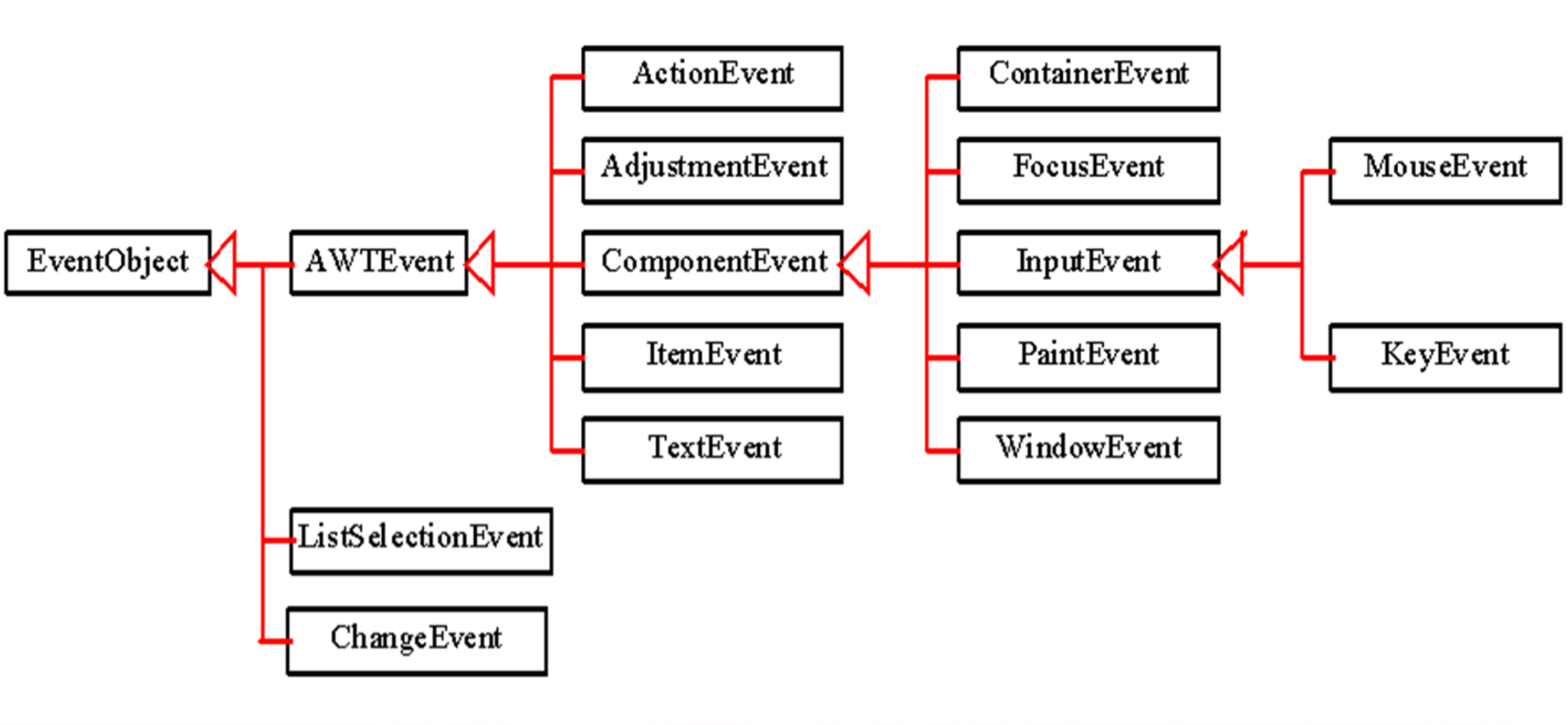
- Τα αντικείμενα-ακροατές είναι **ανώνυμα** αντικείμενα
  - `redButton.addActionListener(new RedListener());`

- Μπορούμε να κάνουμε τον κώδικα ακόμη πιο συνοπτικό ορίζοντας μια **ανώνυμη κλάση**
  - Ο ορισμός της κλάσης γίνεται εκεί που τον χρειαζόμαστε μόνο και υλοποιεί ένα Interface
  - Δεν συνίσταται αλλά μπορεί να το συναντήσετε σε κώδικα που δημιουργείται από IDEs

```
redButton.addActionListener(new ActionListener()
  {
     public void actionPerformed(ActionEvent e){
          redPanel.setBackground(Color.RED);
     }
  }
);
```

Ο ορισμός της κλάσης
Χρησιμοποιούμε το όνομα του interface

Κλάσεις Συμβάντων (Event Classes)



Συσχέτιση Ενεργειών Χρήστη με Events

User Action	Source Object	Event Type generated
Click a button	JButton	ActionEvent
Click a check box	JCheckBox	ItemEvent, ActionEvent
Click a radio button	JRadioButton	ItemEvent, ActionEvent
Press return on a text field	JTextField	ActionEvent
Select a new item	JComboBox	ItemEvent, ActionEvent
Window opened, closed, etc.	Window	WindowEvent
Mouse pressed, released, etc.	Component	MouseEvent
Key released, pressed, etc.	Component	KeyEvent

Συσχέτιση Events με Event Handlers

Event Class	Listener Interface	Listener Methods (Handlers)
ActionEvent	ActionListener	actionPerformed(ActionEvent)
ItemEvent	ItemListener	itemStateChanged(ItemEvent)
WindowEvent	WindowListener	windowClosing(WindowEvent)
		windowOpened(WindowEvent)
		windowIconified(WindowEvent)
		windowDeiconified(WindowEvent)
		windowClosed(WindowEvent)
		windowActivated(WindowEvent)
		windowDeactivated(WindowEvent)
ContainerEvent	ContainerListener	componentAdded(ContainerEvent)
		componentRemoved(ContainerEvent)
MouseEvent	MouseListener	mousePressed(MouseEvent)
		mouseReleased(MouseEvent)
		mouseClicked(MouseEvent)
		mouseExited(MouseEvent)
		mouseEntered(MouseEvent)
KeyEvent	KeyListener	keyPressed(KeyEvent)
		keyReleased(KeyEvent)
		keyTypeed(KeyEvent)

Παράδειγμα Event Handling με ΔΥΟ Listeners

```
    JButton jbtOK = new JButton("OK");
    add(jbtOK);
    // Register listeners
    OKListenerClass listener1 = new OKActionListener();
    jbtOK.addActionListener(listener1);

    OKListenerClass listener2 = new OKMouseListener();
    jbtOK.addMouseListener(listener2);
  }
  public static void main(String[] args) {
    JFrame frame = new HandleEvent();
    …
  }
}
class OKActionListener implements ActionListener {
  public void actionPerformed(ActionEvent e) {
    System.out.println("OK button clicked");
  }
}
class OKMouseListener implements MouseListener {
  public void mouseClicked(MouseEvent arg0) {
    System.out.println("mouseClicked");
  }
  public void mouseEntered(MouseEvent arg0) {
    System.out.println("mouseEntered");
  }
}
```

Listener1 - **ActionListener**

Listener2 - **MouseListener**

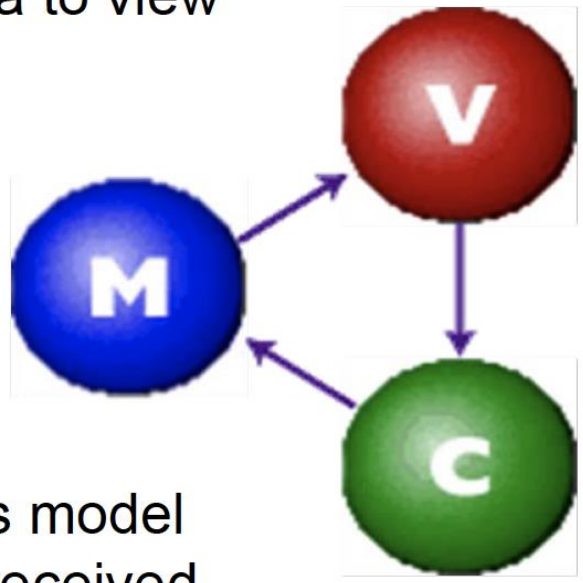
Προβλήματα Κλάσεων Listeners

- Στο προηγούμενο παράδειγμα έχουμε δηλώσει 2 listeners σαν δύο ξεχωριστές κλάσεις
- Οι κλάσεις αυτές θα χρησιμοποιηθούν μόνο μέσα στο συγκεκριμένο πρόγραμμα/κλάση που έχουμε αναπτύξει.
- **Ερώτηση**: Μήπως έχουν και άλλες κλάσεις πρόσβαση σε αυτές τις κλάσεις;
- **Απάντηση**: Δυστυχώς ΝΑΙ! (τουλάχιστον στο ίδιο πακέτο)

- **Ερώτηση**: Μπορώ να δηλώσω μία κλάση listener (ActionListener) που να την χρησιμοποιήσω με όλα τα αντικείμενα που έχω;
- **Απάντηση**: Δυστυχώς ΟΧΙ!
  - Μπορώ όμως να υλοποιήσω το interface ActionListener στο top-level επίπεδο και να διαχειριστώ όλα τα ActionEvents από όλα τα αντικείμενα

Model - View - Controller (MVC)

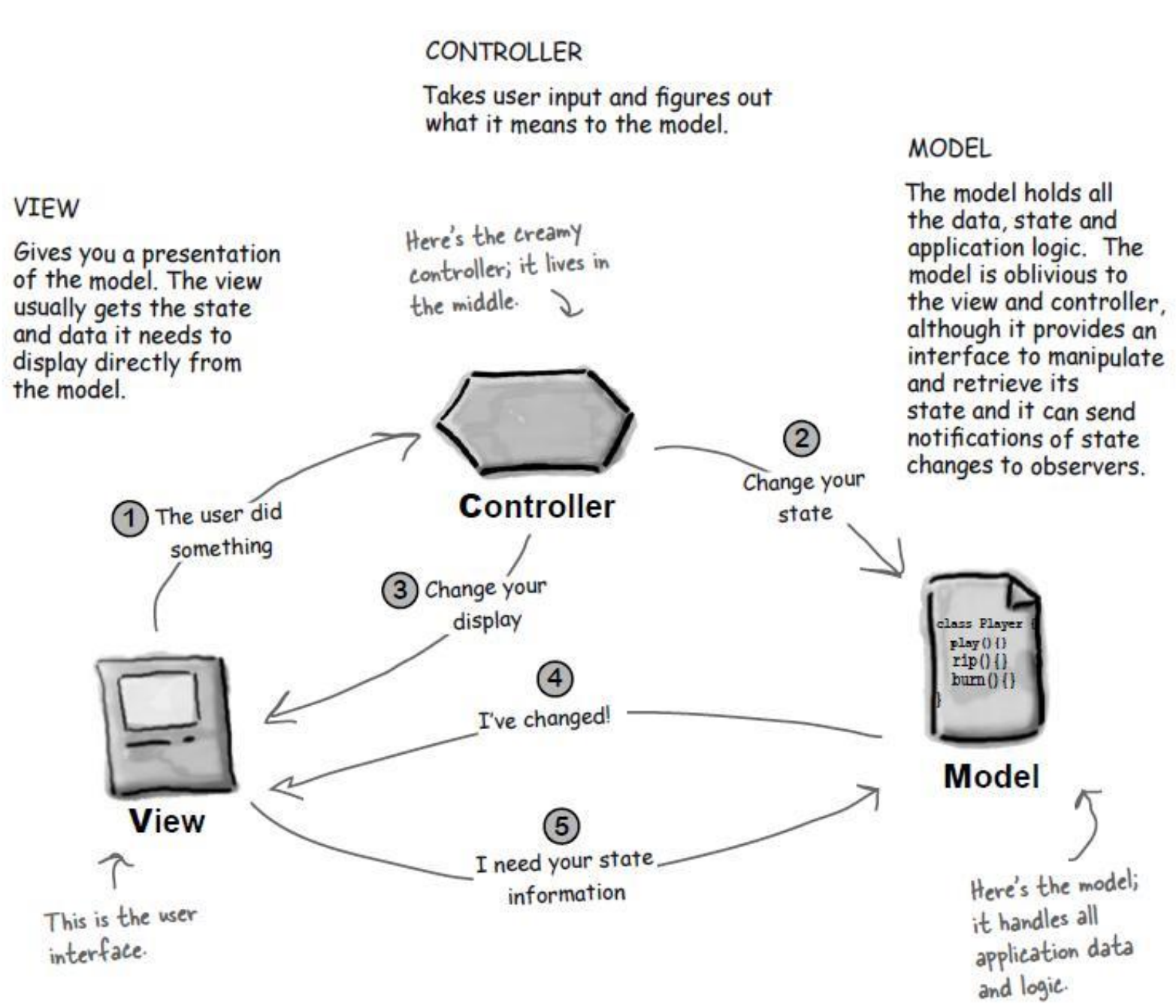
Model passes data to view for rendering



View determines which events are passed to controller

Controller updates model based on events received

Model View Controller (MVC)



Eclipse

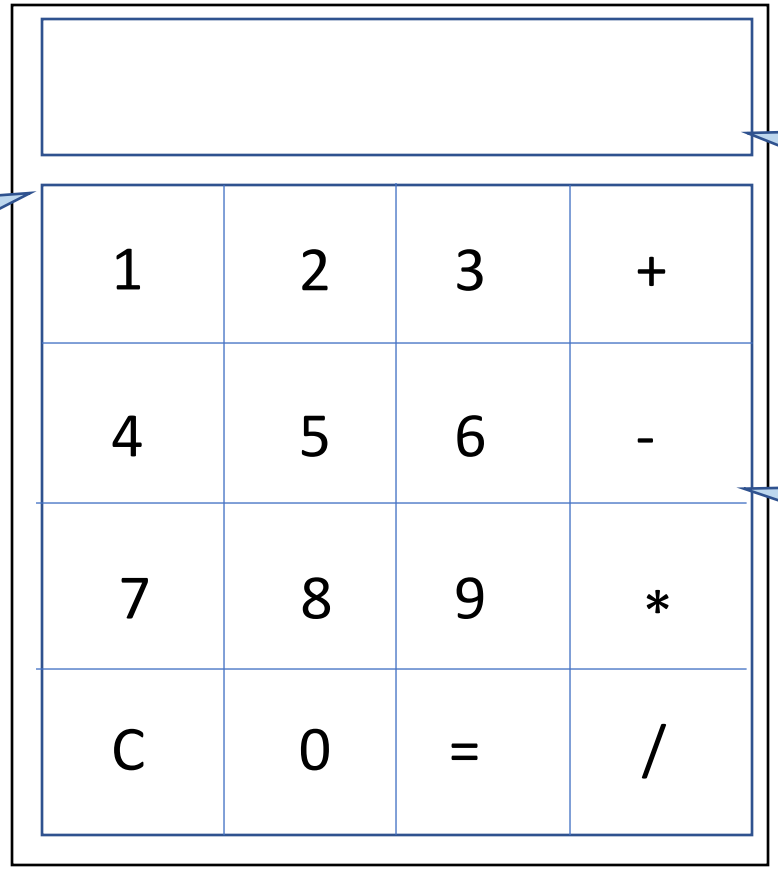
- Η eclipse (αλλά και άλλα IDEs) μας δίνει πολλά έτοιμα εργαλεία για την δημιουργία GUIs
- Εγκαταστήσετε το plug-in **WindowsBuilder**
  - **https://eclipse.dev/windowbuilder/download.php**
  - **https://marketplace.eclipse.org/content/windowbuilder/help**

  - Help -> install new software -> add – archive .. -> (then select both options)

- **Παράδειγμα**: Δημιουργήστε μια αριθμομηχανή

Σχεδιασμός

Τα αποτελέσματα εμφανίζονται στην κορυφή τα πλήκτρα από κάτω



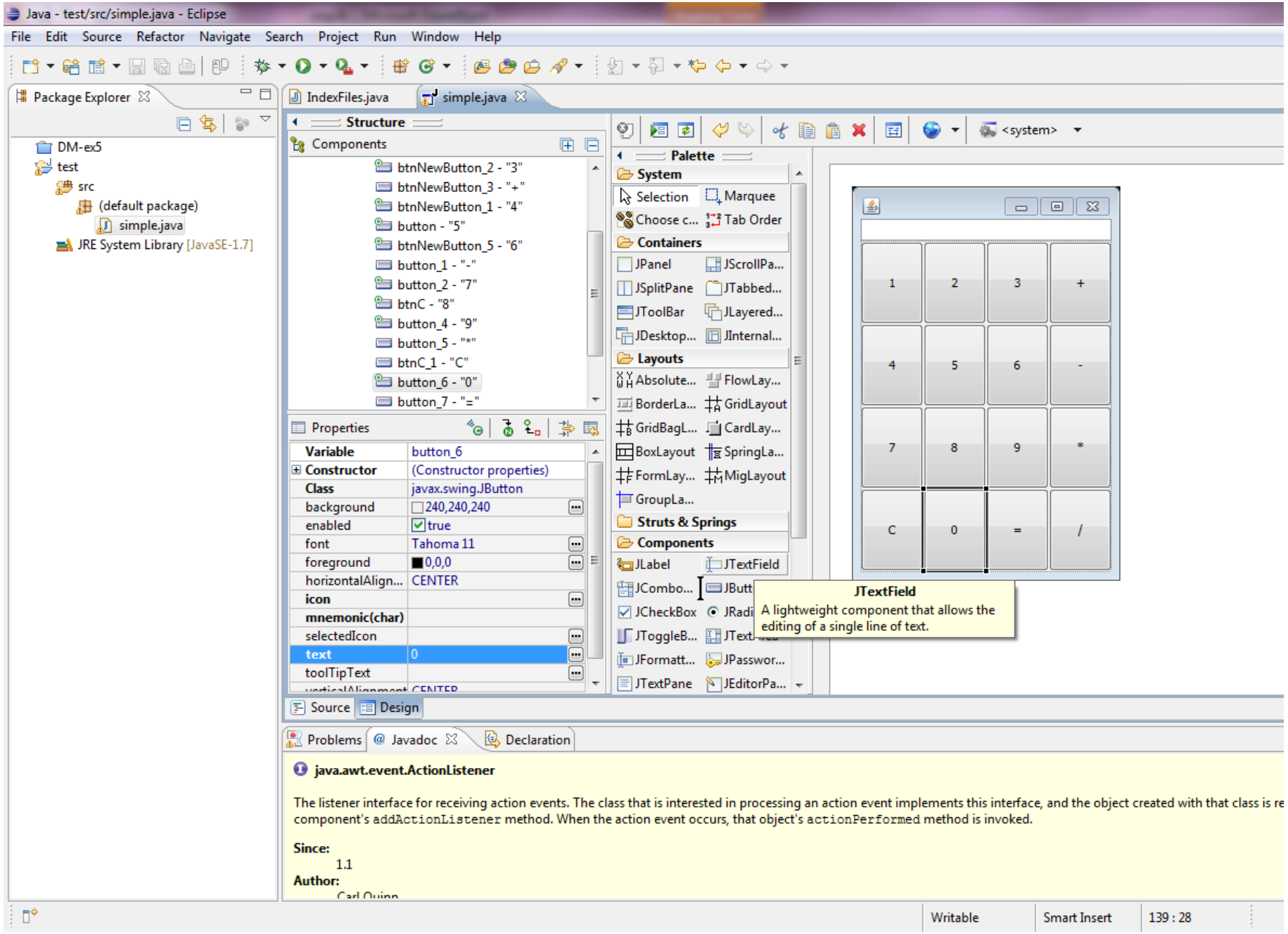
Textbox για να εκτυπώνει το αποτέλεσμα

Κουμπιά για καθένα από τα πλήκτρα

Χρειαζόμαστε ένα border layout για να βάλουμε το textbox στην κορυφή.
Στο κέντρο θα βάλουμε τα κουμπιά. Βάζουμε ένα panel με grid layout

Εισαγωγή μίας διαφάνειας στο Eclipse

- Το Eclipse οργανώνει τον κώδικα σε **projects**
- Οι **κλάσεις** στη συνέχεια προστίθενται μέσα στο project
- Πρέπει να έχετε εγκαταστήσει το plugin **WindowsBuilder**
- Για να φτιάξετε ένα GUI
  - Αρχικά πρέπει να φτιάξετε ένα Java Project
  - Στη συνέχεια προσθέτετε στο project. Επιλέξετε Other>WindowsBuilder>SWING>Application Window
- Στη συνέχεια θα έχετε ένα μενού από τα διάφορα components τα οποία μπορείτε να προσθέτετε στο στην εφαρμογή σας.
  - Μπορείτε να δουλεύετε είτε με το **Design** είτε με τον **Source** κώδικα


Java - test/src/simple.java - Eclipse
File Edit Source Refactor Navigate Search Project Run Window Help
Package Explorer
DM-ex5
test
src
(default package)
simple.java
JRE System Library [JavaSE-1.7]
IndexFiles.java
simple.java
Structure
Components
btnNewButton_2 - "3"
btnNewButton_3 - "+"
btnNewButton_1 - "4"
button - "5"
btnNewButton_5 - "6"
button_1 - "-"
button_2 - "7"
btnC - "8"
button_4 - "9"
button_5 - "*"
btnC_1 - "C"
button_6 - "0"
button_7 - "="
Properties
Variable button_6
Constructor (Constructor properties)
Class javax.swing.JButton
background 240,240,240
enabled true
font Tahoma 11
foreground 0,0,0
horizontalAlign... CENTER
icon
mnemonic(char)
selectedIcon
text 0
toolTipText
Palette
System
Selection Marquee
Choose c... Tab Order
Containers
JPanel JScrollPa...
JSplitPane JTabbed...
JToolBar JLayered...
JDesktop... JInternal...
Layouts
Absolute... FlowLay...
BorderLa... GridLayout
GridBagL... CardLay...
BoxLayout SpringLa...
FormLay... MigLayout
GroupLa...
Struts & Springs
Components
JLabel JTextField
JCombo... JButt
JCheckBox JRadi
JToggleB... JText
JFormatt... JPasswor...
JTextPane JEditorPa...
<system>
1 2 3 +
4 5 6 -
7 8 9 *
C 0 = /
JTextField
A lightweight component that allows the editing of a single line of text.
Source Design
Problems Javadoc Declaration
java.awt.event.ActionListener
The listener interface for receiving action events. The class that is interested in processing an action event implements this interface, and the object created with that class is re
component's addActionListener method. When the action event occurs, that object's actionPerformed method is invoked.
Since:
1.1
Author:
Carl Quinn
Writable
Smart Insert
139 : 28

Δημιουργία κώδικα

- Τα IDEs μας επιτρέπουν να διαχωρίζουμε το **design** από τον **κώδικα**
  - Το πλεονέκτημα είναι ότι έχουμε ένα WYSIWYG interface με το οποίο μπορούμε να σχεδιάσουμε το GUI
  - Το μειονέκτημα είναι ότι δημιουργείται πολύς κώδικας **αυτόματα** ο οποίος δεν είναι πάντα όπως τον θέλουμε.

- Ο διαχωρισμός του σχεδιαστικού κομματιού από τις πράξεις που εκτελούν είναι γενικά μια καλή προγραμματιστική πρακτική.

Δημιουργία κώδικα

- Η δημιουργία ενός κουμπιού δημιουργεί αυτό τον κώδικα

```
JButton button_6 = new JButton("0");
panel.add(button_6);
```

- Αν πατήσουμε πάνω στο κουμπί (double-click) δημιουργείται ο ακροατής του κουμπιού αυτόματα ως μια **ανώνυμη κλάση**

```
JButton button_6 = new JButton("0");
button_6.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
    }
});
panel.add(button_6);
```

Δημιουργία κώδικα

- Η δημιουργία ενός κουμπιού δημιουργεί αυτό τον κώδικα

```
JButton button_6 = new JButton("0");
panel.add(button_6);
```

- Αν πατήσουμε πάνω στο κουμπί (double-click) δημιουργείται ο ακροατής του κουμπιού αυτόματα ως μια **ανώνυμη κλάση**
  - Εμείς συμπληρώνουμε τον κώδικα

```
JButton button_6 = new JButton("0");
button_6.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        textField.setText(textField.getText()+"0");
    }
});
panel.add(button_6);
```